id: pqart-source-docs-secondary-development-pqart二次开发入门 title: PQArt二次开发入门 status: active tags: [PQArt, 二次开发, 入门] source: pq-skill/references/source-docs/secondary-development/PQArt二次开发入门.md updatedAt: 2026-05-09T10:21:40.781Z

PQArt二次开发入门

文档版本：V4.4

修订时间：2025.01.10

产权说明

注意：此处包含的所有信息均为北京华航唯实机器人科技股份有限公司的财产。未经北京华航唯实机器人科技股份有限公司的事先书面许可，本出版物的任何部分（无论是硬拷贝还是电子形式）均不得复制、存储在检索系统中，或以任何形式或通过任何方式如电子、机械、影印、录音、或以其他方式传输。请注意，即使未与包含最终用户许可协议的软件一起分发，本指南中的内容仍受版权法保护。PQ和PQArt的标志是北京华航唯实机器人科技股份有限公司的注册商标。所有其他商标均为其各自所有者的财产。

本出版物和此处的信息按原样提供，仅供参考，如有更改，恕不另行通知，且不应被视为北京华航唯实机器人科技股份有限公司的承诺。北京华航唯实机器人科技股份有限公司明确声明对可能出现在本指南所载信息中的任何错误或不准确之处不承担任何责任或义务，对本文件不作任何（明示、暗示或法定）保证，并明确表示不承担任何关于适销性、适用于特定目的和不侵犯第三方权利的保证。

简介

PQArt二次开发是指在原有PQArt 企业版产品基础上根据每个用户的具体实际业务需求进行的二次开发。原有PQArt体系结构庞大，功能模块较多，其本质是一款面向整个工业环境的机器人离线编程仿真工具软件。但每个用户都有各自的侧重点及相应的业务需求，原有PQArt并不能满足行业内的所有需求。况且随着工业4.0进程的推进、智能制造的发展，离线编程企业就必须要有良好的应变能力和快速响应市场的能力。PQArt作为该领域的技术领先产品，二次开发的需求也越来越大。那么提供二次开发能力，供每个公司据自己的实际业务需求，定制开发一套适合自己的软件，也是大势所趋。

开发平台

开发平台

支持以下Visual Studio版本。

Visual Studio 2015

Visual Studio 2019

Visual Studio 2022

开发语言

支持标准C++、C#语言开发。

软件平台

PQArt系列产品 9.0.0.5981(x64)及以上版本

编译配置

x64 Release

二次开发

PQArt二次开发需要用户按照一定的UI协议定制功能区界面、接口协议开发动态链接库，置于指定目录下，启动PQArt即可加载用户二次开发功能。

功能模块文件

1. 每一个二次开发模块至少需要2个组成部分。

- UI入口文件，后缀为.rbt(c++)或.rbc(C#)。（必需提供）
- 实现功能的动态链接库文件（.dll）。（C#需要使用.Net FrameWork）（必需提供）

1. 每一个二次开发模块都需要以子目录结构放置于指定目录PQExternal下。在软件界面二次开发示例页签下单击二次开发目录按钮即可打开该PQExternal目录。(一般位于：C:\Users\Public\Documents\CHL\PQArtEnt\PQExternal) 其中C++的位于子目录PQExternalSample，C#的位于子目录PQCSharpSample

UI定制

```
PQArt功能区为PQArt功能模块入口按钮放置区域。功能区层级结构为页签(Tab)->面板(Panel)->按钮(Button)，
```

二次开发通过定制.rbt或.rbc文件，将用户功能添加到PQArt功能区。下图-1为一个rbt文件样式以及它在PQArt功能区上显示结果。

图-1 PQArt功能区局部示意图

一个rbt或rbc文件必须是一个完整的xml结构文档，根节点必须为Tab（页签）并且有且只有一个，一个Tab下可以有多个Panel（面板），一个Panel下可以有多个Button（按钮）。

C++ rbt 编写规则

详细规则如下：

Tab：

Label：页签标题

Position：决定该页签放置于PQArt系统页签之前还是之后，“Front”为前，“Back”为后

Panel：

Label：面板标题

Button：

Command：具体功能的ID，为英文字符且唯一不重复

Label：具体功能名称

ToolTip：按钮悬停提示

Icon：按钮图标，不提供将使用默认按钮图标

Action：实现了该功能的动态链接库名称（非绝对路径）

C# rbc 编写规则

详细规则如下：

Tab：

Label：页签标题

Position：决定该页签放置于PQArt系统页签之前还是之后，“Front”为前，“Back”为后

Panel：

Label：面板标题

Button：

Command：需要调用的目标函数，为类的成员函数函数名（目前支持静态函数）

Label：具体功能名称

ToolTip：按钮悬停提示

Icon：按钮图标，不提供将使用默认按钮图标

Action：实现了该功能的C#动态链接库名称（非绝对路径），您可以使用多个DLL

Class：命名空间名.类名，用于确定目标函数所在的命名空间和类

C++ 动态链接库

在PQArt二次开发中需要用到2个接口：IRoExternalApp，IRoDataExchange

其中IRoExternalApp 定义了二次开发与PQArt交互的函数，比如向PQArt注册二次开发命令，执行PQArt系统命令等。IRoDataExchange定义了与PQArt数据交互的函数，比如创建零件、更新场景等。

动态链接库制作规则如下：

1、每一个rbt文件中定义的Button必须要有一个类来实现该按钮的功能，该类要从抽象类CRoExternalCommand派生，在类中实现抽象类的各个虚函数。其中二次开发功能的入口需要实现在Start函数中。

2、动态链接库必须要实现pqapp、pqdata2个导出函数，pqapp传入IRoExternalApp指针，pqdata传入IRoDataExchange指针。

```
3、需要调用IRoExternalApp接口的RegisterExternalCommand函数将按钮功能实现类对象注册给PQArt。
```

MFC工程

参考示例工程在C:\Users\Public\Documents\CHL\PQArtEnt\PQExternal\PQExternalSample\PQExternalSample,直接运行Visual Studio 2015打开PQExternalSample.sln即可。

现以Visual Studio 2015为例，讲解动态链接库的实现。

1、启动Visual Studio 2015，单击New Project， 新建一个MFC dll工程。

2、将PQExternalSample目录下的include目录拷贝至新建项目.vcxproj文件目录下，将include路径设置到工程附加包含目录。

3、新建命令类，从CRoExternalCommand派生，实现CRoExternalCommand的虚函数，并在Start中实现功能入口。

//.h class CCmdOrder : public CRoExternalCommand { public: CCmdOrder(); ~CCmdOrder(); //overwrite public: virtual void Start(); virtual void Run(); virtual void End(); //return FALSE virtual BOOL IsSupportContext(); }; //.cpp void CCmdOrder::Start() { AFX_MANAGE_STATE(AfxGetStaticModuleState()); CCmdOrderDlg dlg; dlg.DoModal(); }

4、给Dll添加2个导出函数pqapp、pqdata。pqapp传入IRoExternalApp接口指针，pqdata传入IRoDataExchange接口指针。

导出函数可以在工程的.def中添加，也可以按以下方式添加。

```
#include "IdmDataExchange.h"
#include "IfmExternalApp.h"
extern "C" {
__declspec(dllexport)
void pqapp(IRoExternalApp* i_spApp);
__declspec(dllexport)
void pqdata(IRoDataExchange* i_spData);
}
5、调用IRoExternalApp接口的RegisterExternalCommand函数注册功能实现类对象和功能ID。
```

// dll .h class CTeaBarSystemApp : public CWinApp { public: CTeaBarSystemApp(); // Overrides public: virtual BOOL InitInstance(); void RegisterPQExternalApp(IRoExternalApp* i_spExternalApp); void RegisterPQDataExchange(IRoDataExchange* i_spDataExchange); public: IRoExternalApp* m_spPQApp; IRoDataExchange* m_spPQDataExc; CCmdOrder *m_cmdOrder;*

DECLARE_MESSAGE_MAP() }; // dll .cpp CTeaBarSystemApp theApp; void pqapp(IRoExternalApp i_spApp) { theApp.RegisterPQExternalApp(i_spApp); } void pqdata(IRoDataExchange* i_spData) { theApp.RegisterPQDataExchange(i_spData); } BOOL CTeaBarSystemApp::InitInstance() { CWinApp::InitInstance(); m_cmdOrder = new CCmdOrder; return TRUE; } void CTeaBarSystemApp::RegisterPQExternalApp(IRoExternalApp* i_spExternalApp) { m_spPQApp = i_spExternalApp; m_spPQApp->RegisterExternalCommand("CMD_OEDER", m_cmdOrder); } void CTeaBarSystemApp::RegisterPQDataExchange(IRoDataExchange* i_spDataExchange) { m_spPQDataExc = i_spDataExchange; }*

6、在dll中添加对话框

在工程上右键选择Class Wizard

单击Add Class，下来列表中选择MFC Class

在弹出的对话框中选择基类为CdialogEx，指定类名称即可新建一个对话框类。

7、实现具体功能，编译，将生成的dll文件、rbt文件、图标文件以及其他文件归档在一个文件夹下，将此文件夹放置于C:\Users\Public\Documents\PQExternal下，重启PQArt即可使用二次开发功能。

QT工程

以下内容以QT5.12.8为例。针对不同版本的PQArt软件，所需QT版本不同，PQArt软件安装包名称中带_QT5字样的需要QT5.12.8，不带_QT5字样的需要QT4.8.7。

1、新建QT dll工程

Qt moudle选择Widgets。

2、删除默认生成的导出类。

3、规划自己的功能，按需添加命令类，命令类需从CRoExternalCommand派生，并实现所以虚函数。

//.h #ifndef CMDQT_H #define CMDQT_H #include "include/IfmExternalApp.h" class CmdQT : public CRoExternalCommand { public: CmdQT(); ~CmdQT(); public: // virtual void Start(); virtual void Run(); virtual void End(); virtual BOOL IsSupportContext() { return FALSE; } }; #endif // CMDQT_H //.cpp #include "CmdQT.h" #include "QTDialog.h" CmdQT::CmdQT() { } CmdQT::~CmdQT() { } void CmdQT::Start() { CQTDialog *qdlg = new CQTDialog(); qdlg->show(); } void CmdQT::Run() { } void CmdQT::End() { }

4、如需添加UI界面，步骤如下

新建一个设计师模板类，设计界面及业务逻辑。

5、将3.3.1提及的PQExternalSample目录下的include目录拷贝至新建项目.pro文件目录下将，在工程默认生成的*******_global.h中添头文件引用，声明导出函数pqapp、pqdata。

ifndef EXTERNALDEV_GLOBAL_H #define EXTERNALDEV_GLOBAL_H #include #include "include/IdmDataExchange.h" #include "include/IfmExternalApp.h" #if defined(EXTERNALDEV_LIBRARY) # define EXTERNALDEV_EXPORT Q_DECL_EXPORT #else # define EXTERNALDEV_EXPORT Q_DECL_IMPORT #endif extern "C" { EXTERNALDEV_EXPORT void pqapp(IRoExternalApp* i_spApp); EXTERNALDEV_EXPORT void pqdata(IRoDataExchange* i_spData); } #endif // EXTERNALDEV_GLOBAL_H

6、新建C++类CPQExternalUtility,管理第3步创建的命令类，负责这些命令类的创建和销毁。调用RegisterExter

//.h #ifndef CPQEXTERNALUTILITY_H #define CPQEXTERNALUTILITY_H #include "ExternalDev_global.h" #include "CmdQT.h" class CPQExternalUtility { public: CPQExternalUtility(); ~CPQExternalUtility(); public: void InitInstance(); void ExitInstance(); void RegisterExternalApp(IRoExternalApp* i_spApp); void RegisterDataExchange(IRoDataExchange* i_spData); IRoExternalApp* GetExternalApp(); IRoDataExchange* GetDataExchange(); protected:

IRoDataExchange *m_spDataExc; IRoExternalApp m_spExtApp; CmdQT m_cmdQT; }; #endif // CPQEXTERNALUTILITY_H //.cpp #include "PQExternalUtility.h" CPQExternalUtility::CPQExternalUtility() { } CPQExternalUtility::~CPQExternalUtility() { } void CPQExternalUtility::InitInstance() { m_cmdQT = new CmdQT(); } void CPQExternalUtility::ExitInstance() { if(m_cmdQT) { delete m_cmdQT; m_cmdQT = NULL; } } void CPQExternalUtility::RegisterExternalApp(IRoExternalApp i_spApp) { m_spExtApp = i_spApp; m_spExtApp->RegisterExternalCommand("CMD_GETROBOTINFO", m_cmdQT); } void CPQExternalUtility::RegisterDataExchange(IRoDataExchange i_spData) { m_spDataExc = i_spData; } *IRoExternalApp* CPQExternalUtility::GetExternalApp() { return m_spExtApp; } IRoDataExchange* CPQExternalUtility::GetDataExchange() { return m_spDataExc; }

7、新增main.cpp，重写DllMain函数，在DLL_PROCESS_ATTACH中创建CPQExternalUtility*对象g_pqUtility，在D

include "ExternalDev_global.h" #include "PQExternalUtility.h" CPQExternalUtility* g_pqUtility; void pqapp(IRoExternalApp* i_spApp) { g_pqUtility->RegisterExternalApp(i_spApp); } void pqdata(IRoDataExchange* i_spData) { g_pqUtility->RegisterDataExchange(i_spData); } BOOL APIENTRY DllMain(HMODULE hModule,DWORD ul_reason_for_call,LPVOID lpReserved) { switch (ul_reason_for_call) { case DLL_PROCESS_ATTACH: { g_pqUtility = new CPQExternalUtility(); g_pqUtility->InitInstance(); break; } case

DLL_THREAD_ATTACH: case DLL_THREAD_DETACH: { break; } case DLL_PROCESS_DETACH: { g_pqUtility->ExitInstance(); if(g_pqUtility) { delete g_pqUtility; g_pqUtility = NULL; } break; } } return TRUE; }

8、业务逻辑实现完成后，编译出release版本dll，连同rbt文件及相关依赖文件一同放在一个文件夹下，并将该文件夹拷贝至3.1中提及的PQExternal下。重启PQArt软件即可查看效果。

C# 动态链接库

C# 二次开发示例中，提供了一个名为PQInterface.cs的接口文件，其定义了与Art交互的所有接口。

动态链接库的编写规则如下：

（1）每个在rbc中声明的按钮，都要有对应的C# DLL中一个函数去实现它

（2）只要一个C#类实现了一个按钮，那么它必须要实现PQData(IntPtr x)方法，用于初始化二次开发环境。

（3）您的C#DLL的编译产物，需要放在PQExternal文件夹的子文件夹中，并且.rbc文件处于这个子文件夹的根目录下。即，在PQExternal文件夹下，您可以创建一个属于您的新工程文件夹（随意命名，无需配置），例如MyProject。然后在MyProject文件夹下，必须存在：.rbc fmCLR.dll PQCSharpIntreop.dll MyProject.dll 其中

1、

打开 PQCSharpSample 示例，文件夹位于：C:\Users\Public\Documents\CHL\PQArtEnt\PQExternal\PQCSharpSample 打开子文件夹PQCSharpSample中的解决方案PQCSharpSample.sln

2、

解决方案中包含两个C#源文件，PQInterface.cs为接口文件，不建议修改。PQKitClass.cs为示例中实现功能的文件。

基本结构：

namespace PQKitApplication //命名空间，可以任意改名 { public class PQKitClass //类名，可以任意改名 { // 固定实现的初始化函数，不可更改 public static void PQData(IntPtr x) // 实现功能的函数，可以任意改名 public static void OpenRobx() } }

3、

新增按钮

如果我们需要新增按钮，并且放在另一个DLL中，首先创建一个 C# .NET Framework 项目（可以放在任何您需要的文件夹下，后续只需要把DLL输出到二开目录即可）

4、

复制示例中的PQInterface.cs到您的项目中，并在您自己的类中调用方法

假设实现大致如下：

```
using PQCSharpInterface;
namespace AnotherDLL {
public
class AnotherClass {
// 此函数为固定实现，不可更改，用于初始化环境 public static
void PQData(IntPtr x) {
PQInterface.initpqdata(x);
}
// 此函数为您逻辑的入口，可以改名 public static
void NewButtonImpl() {
string filePath = "D:\\EFFORT-18.robx";
PQInterface.Doc_open_robx(filePath);
}
}
}
```

然后在项目上右键，并点击属性（注意不是在解决方案上右键）

在“生成”中配置DLL文件的输出位置，使其指向Art二开的C#目录（您新建的目录）

5、

在.rbc中配置您新的按钮，您可以把示例的rbc复制过去并稍作修改

PQKitClass 结构：

（假设您的新工程输出的DLL叫做 AnotherDLL.dll）

```
NewButtonImpl 新按钮 新按钮 new-button.png AnotherDLL.dll AnotherDLL.AnotherClass
```

6、

复制fmCLR.dll PQCSharpInterop.dll 到新的文件夹中

必须复制，因为PQCSharpInterop.dll提供了被您调用的接口（如Doc_open_robx），而fmCLR.dll来调用您的函数

7、

启动 Art 开始使用您的功能

接口调用范例

获取机器人

获取所有PQArt场景中的机器人名称和ID信息，ID是二次开发与PQArt进行数据交互的身份标识。

HRESULT Doc_get_obj_bytype(LONG i_lType, VARIANT* o_sNames, VARIANT* o_sIDs);	
Parameters：	
i_lType	指定对象类型，类型值详见下表
o_sNames	所有指定类型对象的名字
o_sIDs	所有指定类型对象的ID

数据类型定义

ObjType	ID
机器人	32
工件	16
工具	48
底座	64
轨迹	80
场景文件	225
状态机	240
坐标系	176

调用示例。

```
VARIANT vNamePara;
vNamePara.parray = NULL;
VARIANT vIDPara;
vIDPara.parray = NULL;
if (NULL ==
theApp.m_spPQDataExc) {
return;
}
theApp.m_spPQDataExc->Doc_get_obj_bytype(32, &vNamePara, &vIDPara);
if (NULL == vNamePara.parray || NULL == vIDPara.parray) {
return;
}
//缓存指定对象名称 BSTR* bufName;
std::vector<std::wstring> vName;
long lenName = vNamePara.parray->rgsabound[0].cElements;
SafeArrayAccessData(vNamePara.parray, (void**)&bufName);
for (int i = 0; i < lenName; i++) { vName.push_back(bufName[i]); }
SafeArrayUnaccessData(vNamePara.parray); //缓存指定对象ID ULONG* bufID; std::vector<ULONG> vID; lon
SafeArrayAccessData(vIDPara.parray, (void**)&bufID);
for (int i = 0; i < lenID; i++) { vID.push_back(bufID[i]); }
SafeArrayUnaccessData(vIDPara.parray);
```

获取机器人下轨迹组

获取PQArt场景中指定机器人下的所有轨迹组名称。

HRESULT Doc_get_pathgroup_name(ULONG i_ulRobotID, VARIANT* o_varNameArray);	
Parameters：	
i_ulRobotID	指定机器人对象的ID
o_varNameArray	返回指定机器人的所有轨迹组名称

调用示例

```
VARIANT vNamePara;
vNamePara.parray = NULL;
theApp.m_spPQDataExc->Doc_get_pathgroup_name(uID, &vNamePara);
if (NULL == vNamePara.parray) {
//判空处理 return;
}
//缓存轨迹组名称 BSTR* bufName;
std::vector<std::wstring> vName;
long lenName = vNamePara.parray->rgsabound[0].cElements;
SafeArrayAccessData(vNamePara.parray, (void**)&bufName);
for (int i = 0; i < lenName; i++) { vName.push_back(bufName[i]); }
SafeArrayUnaccessData(vNamePara.parray);
```

仿真轨迹组

仿真PQArt场景中指定机器人下的指定轨迹组。二次开发通过接口IRoExternalApp调用RunCommand方法并传入相关参数来实现轨迹组仿真。

调用示例如下，i_CmdKey传入RO_CMD_SIMULATE_PATH_GROUP，i_sParam传入轨迹组名称，i_varInitParam的bstrVal传入轨迹组所属的机器人名称。

```
//需提前获取轨迹组名称
CString csGroupName;
CComVariant vPara(csRobotName);
```

```
theApp.m_spPQApp->RunCommand("RO_CMD_SIMULATE_PATH_GROUP", NULL, NULL, CT2A(csGroupName), vPara, NU
```